

Bazele Programării pe Obiecte

C 11-12

***Subprograme -
Proceduri și funcții***



Proceduri și funcții

- *Subprogramele (proceduri și funcții) sunt utile pentru modularizarea codului. Un subprogram grupează o anumită secvență de cod într-un bloc (de forma Sub... End Sub, respectiv Function... End Function), pentru a îndeplini o sarcină specifică.*
- *Diferența dintre proceduri și funcții constă în faptul că funcțiile returnează valori, procedurile nu.*
- *Avantaje:*
 - *reducerea duplicării codului*
 - *reutilizarea codului*
 - *descompunerea unei probleme complexe în subprobleme mai simple*
 - *îmbunătățirea clarității / lizibilității codului*

Proceduri

- *O procedură efectuează o acțiune / prelucrare, fără a returna o valoare apelantului.*

- *Definire – constă în antetul și corpul subprogramului:*

Sub NumeProc (listaParametri)

‘ declaratii si instructiuni

End Sub

unde NumeProc este numele procedurii (un identificator), iar listaParametri este lista parametrilor formali

- *Apelul unei proceduri:*

NumeProc (listaArgumente)

Funcții (1)

- *O funcție efectuează o prelucrare și returnează o valoare apelantului.*
- *Definire – constă în antetul și corpul funcției:*

```
Function NumeFct (listaParametri) As returnType  
    ' declaratii si instructiuni  
    Return returnExp  
End Function
```

unde NumeFct este numele funcției, listaParametri este lista parametrilor formali, returnType este tipul valorii returnate funcția și returnExp valoarea returnată de funcție.

Funcții (2)

- *Returnarea valorii se face printr-o instrucțiune Return.*

Return returnExp

- *Valoarea returnată (returnExp) trebuie să corespundă tipului returnat (returnType) sau cel puțin să fie convertibilă la acesta. Instrucțiunea Return este ultima instrucțiune executată înainte ca execuția să revină apelant.*

- *Apelul unei funcții are sintaxa:*

NumeFct (listaArgumente)

- *Fiecare argument din lista de argumente din apel trebuie să corespundă parametrului corespunzător din definiția funcției. Pentru a utiliza valoarea returnată de funcție, de obicei apelul este inclus într-o expresie.*

Parametri și argumente

- *Un **parametru** reprezintă o valoare pe care un subprogram (procedură / funcție) se așteaptă să o primească atunci când este apelat. La declararea subprogramului se specifică parametrii acestuia. Pentru fiecare parametru, trebuie precizat numele (identificatorul), tipul de date și mecanismul de transmitere (ByVal sau ByRef).*
- *Prin **argument** se înțelege valoarea furnizată unui parametru atunci când se apelează procedura / funcția. La apelul subprogramului trebuie furnizată lista de argumente. Fiecare argument corespunde parametrului aflat pe aceeași poziție în lista de parametri dată la declarare. Tipul fiecărui argument trebuie să corespundă tipului declarat pentru parametrul corespunzător sau trebuie să fie convertibil la acesta.*

Transferul parametrilor (1)

Transmiterea prin valoare

- În cazul transmiterii prin valoare, parametrii formali ai unei funcții / proceduri sunt copii ale valorilor argumentelor. Aceasta înseamnă că subprogramul nu va modifica valorile argumentelor de apel. Argumentele date la apel pot fi expresii, constante, variabile. Este modul implicit de transfer.
- Parametrii sunt declarați cu ByVal:

```
Sub swap(ByVal x As Integer, ByVal y As Integer)
    Dim aux As Integer
    aux = x
    x = y
    y = aux
End Sub
```

Transferul parametrilor (2)

Transmiterea prin referință

- *Dacă un subprogram trebuie să modifice valoarea unui argument de apel, parametrul respectiv trebuie transmis prin referință. În acest caz, parametrii formali ai funcției / procedurii sunt referințe ale celor actuali, iar toate modificările făcute asupra parametrilor formali se fac de fapt asupra celor actuali. Argumentele de apel pot fi doar variabile sau expresii variabile*
- *Parametrii sunt declarați cu ByRef.*

```
Sub swap(ByRef x As Integer, ByRef y As Integer)
    Dim aux As Integer
    aux = x
    x = y : y = aux
End Sub
```


Parametri opționali

- Dacă un parametru este definit opțional, argumentul corespunzător lui poate fi omis în momentul apelului, intrând în calcul cu valoarea implicită. Pot fi omise și argumente din mijlocul listei de argumente.
- Parametrii opționali trebuie definiți la sfârșitul listei de parametri și este obligatoriu ca aceștia să aibă valori implicite.
- Parametrii sunt declarați cu *Optional*.

```
Function ConcatString (s1 As String,  
    Optional sep As String =vbCrLf, Optional s2 As String = "")  
    Return s1 + sep + s2  
End Function
```

- *Exemple apeluri:* ConcatString(TB1.Text, , TB2.Text)
ConcatString(TB1.Text, " ", TB2.Text)

Parametri variabili (1)

- *Dacă se dorește definirea unui subprogram care să nu aibă un număr fix de parametri, se poate utiliza un ParamArray pentru gestionarea parametrilor variabili, după următoarele reguli:*
 - *un subprogram poate avea un singur ParamArray, iar acesta trebuie definit ultimul în lista parametrilor*
 - *parametrul ParamArray este transmis prin valoare și implicit este opțional*
 - *toți ceilalți parametri ai subprogramului sunt obligatorii*
- *La apel, argumentul corespunzător ParamArray poate fi:*
 - *poate lipsi, caz în care funcția primește un array gol*
 - *o listă de argumente (valori) arbitrare (separate prin virgulă) de tipul elementelor ParamArray-ului (sau convertibile la acesta)*
 - *un array cu elemente de tipul elementelor ParamArray-ului*

Parametri variabili (2)

```
Function SumaNumere(ParamArray numere As Integer())  
    Dim suma As Integer  
    For Each numar As Integer In numere  
        suma += numar  
    Next  
    Return suma  
End Function
```

- *Exemple apeluri:*

```
TextBox1.Text = SumaNumere()  
TextBox1.Text = SumaNumere(12, 23, 10, -10, 5)  
Dim numere() As Integer = {12, 23, 10, -10, 5}  
TextBox1.Text = SumaNumere(numere)
```

Supraîncărcare (1)

- *Supraîncărcarea funcțiilor / procedurilor permite definirea mai multor funcții / proceduri cu același nume – atunci când se dorește definirea mai multor funcții cu același scop, dar care se aplică pe liste diferite de parametri. Funcțiile / procedurile trebuie să difere ca număr și / sau tip al parametrilor. Valoarea returnată nu constituie un criteriu de individualizare.*
- *Identificarea variantei de apelat se face în funcție de numărul, ordinea și tipul argumentelor de la apel. Dacă se identifică o funcție unică ce corespunde, căutarea se oprește și funcția respectivă se apelează. Dacă se identifică mai multe, se va semnala eroare de ambiguitate. Dacă nu se identifică niciuna, se va semnala eroare de linkeditare.*

Supraîncărcare (2)

```
Function SumaNumere(a As Integer, b As Integer) As Integer  
    Return a + b
```

```
End Function
```

```
Function SumaNumere(a As Double, b As Double) As Double  
    Return a + b
```

```
End Function
```

```
Function SumaNumere(a As Integer, b As Integer, c As Integer)  
    Return a + b + c
```

```
End Function
```

- *Exemple apeluri:*

```
TextBox1.Text = SumaNumere(12, 13)
```

```
TextBox1.Text = SumaNumere(12, 13, 14)
```

```
TextBox1.Text = SumaNumere(12.2, 13)
```

Recursivitate

- *Un subprogram recursiv este o funcție / procedură care se autoapelează direct sau indirect.*

```
Function Factorial (n As Integer) As Integer
```

```
    If n <= 1 Then Return 1 Else Return Factorial(n-1)*n
```

```
End Function
```

- *Un subprogram recursiv trebuie să implementeze o condiție limitativă: să includă cel puțin o situație în care recursivitatea se va opri, iar execuția să conducă către acea situație.*
- *Într-o implementare recursivă trebuie avută în vedere și eficiența. Orice algoritm recursiv poate fi implementat și iterativ, posibil cu performanța mai bune.*



END